

Spécifications techniques

Menu Maker by Qwenta

Version	Auteur	Date	Approbation
1.0	Nadia CHOUIREF	03/08/2026	Soufiane

Introduction	2
I.Choix technologiques.....	2
II.Liens avec le back-end.....	6
III.Préconisations concernant le domaine et l'hébergement.....	6
IV. Accessibilité.....	6
V.Recommandations en termes de sécurité	7
VI. Maintenance du site et futures mises à jour	8

Introduction

Conformément aux spécifications fonctionnelles transmises par Qwenta, le site devra respecter :

- La maquette Figma fournie ;
- La palette de couleurs imposée : beige (#FFF4E8), vert (#8BC7B1), noir (#000), blanc (#FFF), marron (#C5A073).

Par ailleurs, aucune stratégie SEO particulière n'est requise à ce stade, et aucun outil d'analyse du comportement utilisateur (type Google Analytics) n'est prévu pour le moment.

I. Choix technologiques

Besoin	Contraintes	Solution	Description de la solution	Justification (2 arguments)
Connexion par e-mail (sans mot de passe)	Un simple lien de confirmation envoyé par e-mail (voir maquette login)	Node.js/Express (génération de token JWT à courte durée) Resend/SendGrid pour l'envoi d'e-mail"	Librairie qui gère l'authentification par lien magique envoyé par e-mail, sans mot de passe	1) Évite de gérer un système de mot de passe (sécurité simplifiée) 2) Solution standard, bien maintenue, compatible React
Création de catégories de menu	Ajout via une modale (voir maquette "Category_overlay")	react-modal	Librairie React pour créer des fenêtres modales accessibles	1) Cohérent avec le choix de React 2) Gère nativement l'accessibilité clavier

Besoin	Contraintes	Solution	Description de la solution	Justification (2 arguments)
Ajout de plats (nom, prix, description, photo)	Upload d'image max 2 Mo (voir maquette)	react-dropzone pour l'upload + stockage sur Cloudinary ou AWS S3	react-dropzone gère le glisser-déposer de fichiers ; Cloudinary stocke et optimise les images	1) Cloudinary redimensionne automatiquement les images (performance) 2) Solution gratuite jusqu'à un certain quota, adaptée à un MVP
Personnalisation (police + couleur)	Le rendu doit s'appliquer en temps réel sur l'aperçu du menu	Google Fonts API + gestion d'état React (useState ou Context API)	Charge dynamiquement les polices choisies et applique les styles via le state React	1) Solution gratuite et large choix de polices 2) Pas besoin de librairie tierce, React suffit
Export du menu en PDF	Doit reproduire fidèlement le rendu visuel du menu	react-pdf ou html2canvas + jsPDF	Convertit le rendu HTML/CSS du menu en fichier PDF téléchargeable	1) Fonctionne côté client, pas besoin de serveur dédié 2) Largement utilisé et documenté
Diffusion sur Deliveroo	Le menu doit pouvoir être transmis/synchronisé vers Deliveroo (voir carte "Exportation Deliveroo", priorité	API Deliveroo Partner (ou solution manuelle si pas d'accès API : export au format attendu par Deliveroo)	Permet d'envoyer les données du menu (plats, prix, catégories) directement vers la plateforme Deliveroo	1) Évite une re-saisie manuelle du menu par le restaurateur sur Deliveroo 2) Solution qui suit un standard imposé par

Besoin	Contraintes	Solution	Description de la solution	Justification (2 arguments)
	P2)			Deliveroo, pas de réinvention
Partage sur Instagram	Partage d'une image du menu (voir carte "Partage sur Instagram", priorité P2)	Génération d'une image via html2canvas + Web Share API (ou lien de partage simple)	Convertit le menu en image, puis ouvre une fenêtre de partage native ou propose le téléchargement pour publication manuelle	1) Instagram n'a pas d'API publique simple d'upload direct pour ce cas d'usage, donc solution pragmatique 2) html2canvas déjà utilisé ailleurs (cohérence technique)
Sauvegarde des menus précédemment créés	Le restaurateur doit retrouver ses menus dans "Mes menus" (voir maquette)	Back-end Node.js + PostgreSQL (ou MongoDB selon structure des données)	Stocke les menus créés en base de données, associés au compte du restaurateur	1) Node.js cohérent avec un écosystème JS unifié (front + back en JS) 2) PostgreSQL adapté à des données structurées (menu → catégories → plats), relation claire
Commander l'impression d'un menu	Fonctionnalité "Imprimer un menu" (priorité P1, back-office)	Service tiers d'impression à la demande (ex: API Printful ou solution équivalente) à définir avec	Permet de déclencher une commande d'impression physique du menu depuis le site	1) Qwenta est justement spécialisé impression, donc ce point mérite d'être clarifié avec eux (interne ou prestataire) 2) Évite de développer un

Besoin	Contraintes	Solution	Description de la solution	Justification (2 arguments)
		Qwenta		<i>système d'impression physique en interne, hors périmètre front-end</i>

II. Liens avec le back-end

- Quel langage pour le serveur : *NodeJS (avec Express.js comme framework serveur).*
- A-t-on besoin d'une API : *Oui, plusieurs API sont nécessaires :*
 - Une **API interne** (créée par l'équipe de dev, en Node.js/Express) pour gérer les comptes, menus, plats
 - Des **API externes** : Deliveroo Partner API (diffusion menu), potentiellement une API d'e-mail comme Resend/SendGrid (pour la connexion par lien magique)
- Base de données choisie : *SQL : les données du Menu Maker sont très structurées et relationnelles (un restaurateur → plusieurs menus → plusieurs catégories → plusieurs plats), ce qui correspond bien au modèle relationnel SQL*

III. Préconisations concernant le domaine et l'hébergement

- Nom du domaine : `menumaker.qwenta.com`
- Nom de l'hébergement : Vercel
- Adresses e-mail : `contact@menumaker.qwenta.com`

IV. Accessibilité

- Compatibilité navigateur : Chrome, Safari, Firefox (dernières versions uniquement)
- Types d'appareils : Version **desktop uniquement**

V. Recommandations en termes de sécurité

Point	Solution technique	Justification
Accès aux comptes	Authentification par lien magique envoyé par e-mail (déjà vu en partie I/II, via Resend/SendGrid), pas de mot de passe stocké	Élimine le risque de mots de passe faibles ou piratés — un des risques de sécurité les plus courants
Protection contre les attaques courantes	Respect des recommandations OWASP (protection contre XSS, injections SQL via un ORM comme Prisma, limitation du taux de requêtes/rate limiting)	OWASP est la référence en sécurité web (source de veille, dossier "Dev Web - Général")
Gestion des accès API tierces (Deliveroo, e-mail)	Clés API stockées dans des variables d'environnement (jamais dans le code), gérées via un coffre-fort comme Azure Key Vault ou équivalent	Évite l'exposition de clés sensibles si le code est un jour rendu public ou piraté
Sécurité des uploads (photos de plats)	Limitation stricte du type de fichier (jpg/png uniquement) et de la taille (2 Mo max, déjà précisé dans les specs fonctionnelles)	Empêche l'upload de fichiers malveillants déguisés en image

VI. Maintenance du site et futures mises à jour

Point	Contenu
Suivi des bugs	Mise en place d'un outil de suivi (ex : le tableau Kanban/Notion déjà utilisé pour le développement), avec une colonne dédiée aux anomalies remontées après mise en production
Mises à jour de sécurité	Mise à jour régulière des dépendances (librairies React, Node.js) pour corriger les failles de sécurité connues, via un outil comme Dependabot
Évolutions fonctionnelles	Ajout progressif des fonctionnalités priorité P2/P3 identifiées dans les User Stories (ex : diffusion Deliveroo, partage Instagram, tarifs, mentions légales) une fois le MVP (priorité P1) livré et validé
Hébergement et disponibilité	Surveillance de la disponibilité du site (uptime) via les outils de monitoring intégrés à l'hébergeur (Vercel)
Fréquence de maintenance	À définir avec le client (ex : forfait mensuel d'heures de maintenance, ou intervention à la demande)